NMAM INSTITUTE OF TECHNOLOGY, NITTE

Department of Information Science and Engineering

OPERATING SYSTEMS

PROGRAMMING ASSIGNMENT REPORT

Readers–Writers and Dining Philosophers: Semaphores, Monitors and Program-Generated HTML Simulation

Name: Felicia Fernandes

USN: NNM24IS083

Course: Operating Systems

GitHub Repository: https://github.com/feliciafernandes/os-task-2

SUBMITTED TO

Dr Jason Elroy Martis

Associate Professor

Dept. of Information Science and Engineering, Nitte

1. Problem Statement and Objectives

1.1 Understanding the Problem

If two threads work on the same data at once, the final answer can change depending on which one gets to run first. Leaving that order to chance gives a **race condition**. The program might work on most runs and then fail on an unlucky one. To stop this, the operating system offers synchronization tools, mainly semaphores and monitors, which make threads run in a safe order [1][2].

In this assignment I looked at two classic synchronization problems. The first is **Readers–Writers**, where many threads share one piece of data. Readers can use it together, but a writer has to be on its own. The second is **Dining Philosophers**. Here four philosophers share four forks placed between them, and each one needs both neighbouring forks to eat, so deadlock is a real risk. I solved each problem twice, once with semaphores and once with a monitor. Everything is written in **Python 3** and uses only the standard `threading` library [8].

While each program runs, it keeps a record of what its threads are doing. At the end it writes an HTML page that replays exactly that run in the browser.

1.2 Goals of the Assignment

The main goals were to:

- understand why threads that share data need synchronization, and how it is done
- solve Readers–Writers using semaphores and also using a monitor
- solve Dining Philosophers for four philosophers using semaphores and also using a monitor
- make sure the shared resources are mutually exclusive and show that deadlock cannot happen
- talk about starvation and reduce it where I could
- record every state change while the program runs and build an HTML simulation from that record
- have the program check its own correctness while it runs

2. Readers–Writers: Semaphore-Based Solution

2.1 Background and Purpose

In Readers–Writers, two kinds of threads share one document. Readers only look at it, so any number of them can be inside at the same time. Writers change it, so while a writer is busy no other writer or reader is allowed in [1][4]. The simplest textbook version lets new readers keep coming in as long as one reader is already inside. A steady stream of readers can then keep a writer waiting forever. My goal here was a version that is both safe *and* fair to readers and writers [3].

2.2 Program Design and Development

`readers_writers_semaphore.py` starts 4 reader threads and 2 writer threads by default. Both numbers can be changed from the command line. The shared resource is a `SharedDocument` object, which holds a value, a version number and the name of the last writer. It has no lock of its own. The only protection comes from three `threading.Semaphore` objects:

Semaphore	**Initial value**	**Role**
`service_queue`	1	A turnstile that every thread goes through in the order it arrived, so readers who arrive after a waiting writer can't jump ahead of it
`read_count_mutex`	1	Guards the count of readers that are currently inside
`resource`	1	Gives exclusive access to the document, held either by one writer or by the readers as a group

An event recorder, which is thread-safe, logs every arrival, every entry to and exit from the critical section, and every value that is read or written. Along with each event it saves a snapshot of the whole system.

2.3 Execution Process and Logic

1. Any thread that wants the document first goes through `service_queue`, in the order it arrived.
2. A reader adds one to `read_count` while holding `read_count_mutex`. If it is the first reader in, it locks `resource` on behalf of all the readers.
3. The reader then lets go of the turnstile so other readers can join, and reads the document (critical section).
4. When leaving, the last reader out unlocks `resource`.
5. A writer locks `resource` while it is still holding the turnstile. Then it releases the turnstile and writes on its own (critical section).
6. The writer unlocks `resource` and the next thread in the queue carries on.

Since a waiting writer is holding the turnstile, readers who arrive later have to queue behind it. That way neither readers nor writers can be starved.

3. Readers–Writers: Monitor-Based Solution

3.1 Background and Purpose

A monitor puts the shared data in one place together with the only procedures that are allowed to touch it. Only one thread at a time can be running inside those procedures. If a thread can't continue, it sleeps on a condition variable until another thread signals it [1][5]. Python doesn't have a `monitor` keyword, so I built the same behaviour out of a lock and condition variables [8].

3.2 Program Design and Development

In `readers_writers_monitor.py` the monitor is the `ReadWriteMonitor` class. It has three parts:

- **One lock:** a single `threading.Lock`. Every entry procedure begins with `with self._lock:`, which means only one thread can be inside the monitor at any time.
- **Two condition variables:** `ok_to_read` and `ok_to_write`, both created on that same lock.
- **Four entry procedures:** `start_read`, `end_read`, `start_write` and `end_write`. A thread can only get into the monitor through these.

Conditions in Python follow **Mesa semantics**. When a thread is signalled, it still has to win the lock back before it runs, and by that time the state might have changed. So every wait is written as a `while` loop that checks its condition again after waking up [7].

The shared resource is `SharedDatabase`. It stores a value and a checksum copy of that value. A writer updates the two fields one after the other, so if a reader were let in during a write, it would see two values that don't match. In other words, the data itself would show any race.

3.3 Execution Process and Logic

1. A reader calls `start_read`. It waits on `ok_to_read` if a writer is writing *or* if any writer is waiting. This stops a stream of readers from starving the writers.
2. A writer calls `start_write` and waits on `ok_to_write` until there is no reader and no other writer inside.
3. The actual reading and writing happen *outside* the monitor lock. The monitor only decides who is allowed in, and that is what lets several readers read at once.
4. When the last reader leaves, `end_read` signals one waiting writer.
5. When a writer is done, `end_write` wakes up every reader that was waiting at that moment and lets them all in as one batch, before the other writers. This means writers can't starve readers either. A `write_epoch` counter tells each reader that wakes up whether it is part of that batch.

4. Dining Philosophers: Semaphore-Based Solution

4.1 Background and Purpose

Four philosophers sit around a round table and alternate between thinking and eating. There is one fork between each pair of neighbours, and a philosopher needs both the left and the right fork to eat [1][6]. Suppose every philosopher picks up their left fork at the same moment. Each one then waits forever for a right fork that a neighbour is already holding. This is called **deadlock**. My aim was a semaphore solution where that can never happen.

4.2 Program Design and Development

In `dining_philosophers_semaphore.py` each of the four forks, F0 to F3, is a `threading.Semaphore(1)`. A starting value of 1 means the fork is lying on the table. Philosopher *i* uses F*i* as the left fork and F*(i+1) mod 4* as the right fork, and each philosopher runs in its own thread.

I prevent deadlock with **resource ordering**. Every philosopher always picks up the lower-numbered of their two forks first. For P0, P1 and P2 that is simply left then right, but P3 takes F0 before F3. Because every thread asks for forks in the same global order, it is impossible to get a circle of philosophers each waiting on the next [1]. The program also logs every fork request, every pick-up and put-down, and who owns each fork after every event.

4.3 Execution Process and Logic

1. Each philosopher starts off thinking for a random amount of time.
2. Once hungry, it waits on the semaphore of its lower-numbered fork.
3. Next it waits on the semaphore of its higher-numbered fork.
4. Holding both forks, it eats. Eating is the critical section.
5. It puts both forks back with `release()`, which can wake up a neighbour who was waiting for them.
6. It goes back to thinking, and this repeats until it has eaten the set number of meals.

Starvation: ordering the forks gets rid of deadlock, but it doesn't make things fair, because a Python semaphore doesn't promise to wake the thread that has waited longest. To keep an eye on this, the program prints how many meals each philosopher ate and their longest hungry wait. In the recorded run all four philosophers ate 5 meals, and no one waited longer than 0.22 s.

5. Dining Philosophers: Monitor-Based Solution

5.1 Background and Purpose

With a monitor, forks aren't locked one by one. Instead the monitor keeps track of what each philosopher is doing and gives out both forks at once, and only when both are free [1]. So a philosopher never ends up holding one fork while waiting for the other, which is exactly the step that makes deadlock possible.

5.2 Program Design and Development

The `DiningMonitor` class in `dining_philosophers_monitor.py` is based on the monitor solution in Silberschatz et al. [1]. It is made up of:

- **One lock:** a single `threading.Lock` that protects the monitor.
- **A state array:** `state[i]`, set to THINKING, HUNGRY or EATING for each philosopher.
- **Condition variables:** one `threading.Condition` per philosopher, named `self[i]`, all built on the same lock.
- **Procedures:** the entry procedures `pickup(i)` and `putdown(i)`, plus a private helper `test(i)`.

The monitor also keeps a note of which philosopher owns each fork, which the HTML page uses to draw fork ownership.

The basic monitor can still starve a philosopher if both of their neighbours keep taking turns to eat. To help with this I added an optional ticket rule (`--fair`, which is on by default). It stops a hungry philosopher from overtaking a hungry neighbour who has been waiting for longer.

5.3 Execution Process and Logic

1. Each philosopher thinks for a random amount of time.
2. It calls `pickup(i)`, which sets `state[i] = HUNGRY` and then calls `test(i)`.
3. `test(i)` only lets the philosopher eat if neither neighbour is eating. If so, it changes the state to EATING, hands both forks to the philosopher and signals `self[i]`.
4. If the test fails, the philosopher waits on `self[i]` inside a `while state[i] != EATING` loop.
5. It eats outside the monitor and then calls `putdown(i)`. This puts the state back to THINKING and runs `test()` on both neighbours, so a hungry neighbour can be woken up.

Since both forks are always taken together in one step inside the monitor, no philosopher ever holds just one fork. Hold-and-wait never happens, and so deadlock is impossible.

6. Creating Browser-Based Execution Views

6.1 Background and Purpose

A console log with hundreds of lines is hard to read. That's why the assignment asks each program to produce an HTML page that shows the execution visually. The page has to come from the program's own run, not from a separately drawn animation. All four of my programs do this by building the page out of the events their threads recorded.

6.2 Program Design and Development

Every program goes through these steps:

1. It starts real `threading.Thread` objects, one for each reader, writer or philosopher.
2. The threads only synchronize through the semaphores or the monitor. The random sleeps just stand in for the time spent thinking, eating, reading or writing.
3. Each time a state changes, an `EventRecorder` (which has its own small lock) saves the details below, and the same event is printed to the console:
    - a timestamp in milliseconds
    - which thread it was and what kind of event
    - a full snapshot of every thread's state, the waiting queues, the shared value or the fork owners, and the meal counts
4. Once all threads have finished, the list of events is turned into JSON and placed inside a single self-contained HTML file with inline CSS, JavaScript and SVG.
5. Opening the page in a browser replays the events, with Play/Pause, Step, Reset and a speed slider.

The four HTML files are saved in `generated_html/`. Running a program again gives a new trace, and so a new page.

Each program also checks its safety rules as it runs and prints how many violations it found:

- **Readers–Writers:** a writer is never inside at the same time as anyone else.
- **Dining Philosophers:** two neighbours never eat at the same time.

To make sure these checks actually work, the synchronization was removed or broken on purpose in some test runs. In those runs the checkers reported 20, 37 and 380 violations.

7. Program Runs and Observations

I ran each of the four Python programs separately with `--seed 42`. All of them finished normally and reported **0 invariant violations**. Each screenshot shows the HTML page part of the way through its replay.

7.1 Readers–Writers Using Semaphore

Here writer W2 is writing on its own (value 8 → 17). All four readers and writer W1 are waiting in the order they arrived (R4, R3, R2, W1, R1). R1 arrived after W1, so it can't get ahead of it.

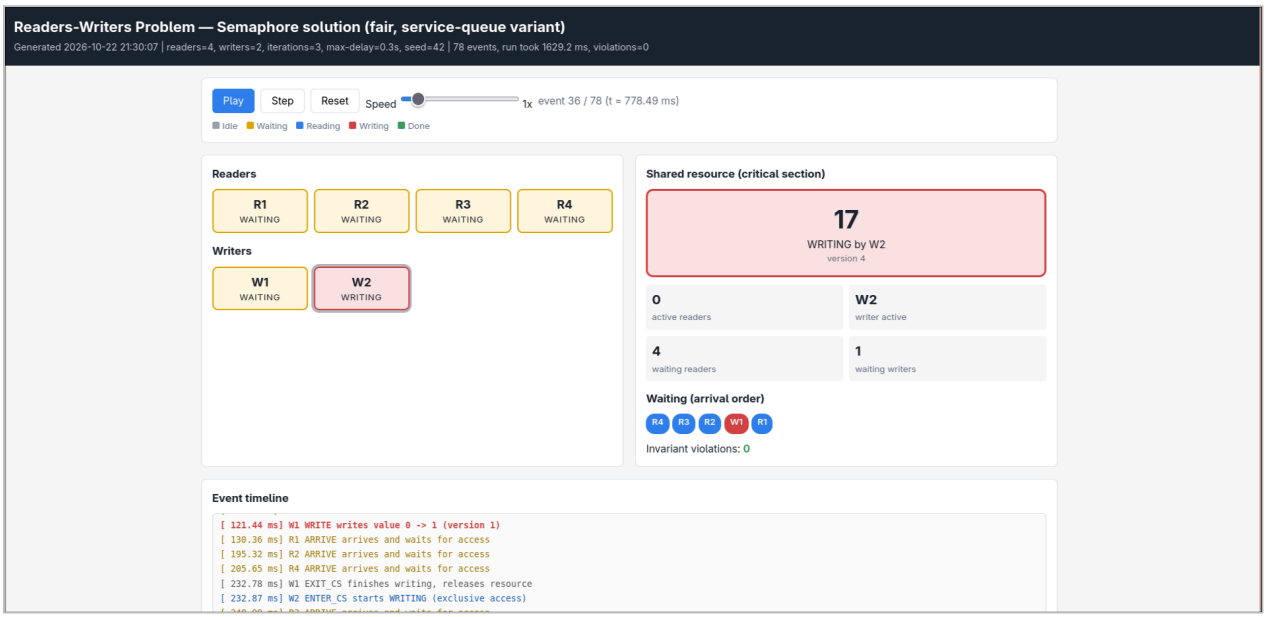


Figure 1. Readers–Writers (semaphore) simulation

```
[648.61 ms] W1 ENTER_CS  starts WRITING (exclusive access)
[688.18 ms] W1 EXIT_CS   finishes writing, releases resource
[688.24 ms] W2 ENTER_CS  starts WRITING (exclusive access)
[699.21 ms] W1 ARRIVE    arrives and waits for access
[750.61 ms] R1 ARRIVE    arrives and waits for access
[778.67 ms] W2 WRITE     writes value 8 -> 17 (version 4)
```

7.2 Readers–Writers Using Monitor

W2 is writing on its own (value 3 → 8). R1 and R3 are waiting on `ok_to_read` and W1 is waiting on `ok_to_write`, while R2, R4 and R5 are already done. In the coloured timeline, the red writing bars never overlap with any other thread reading or writing.

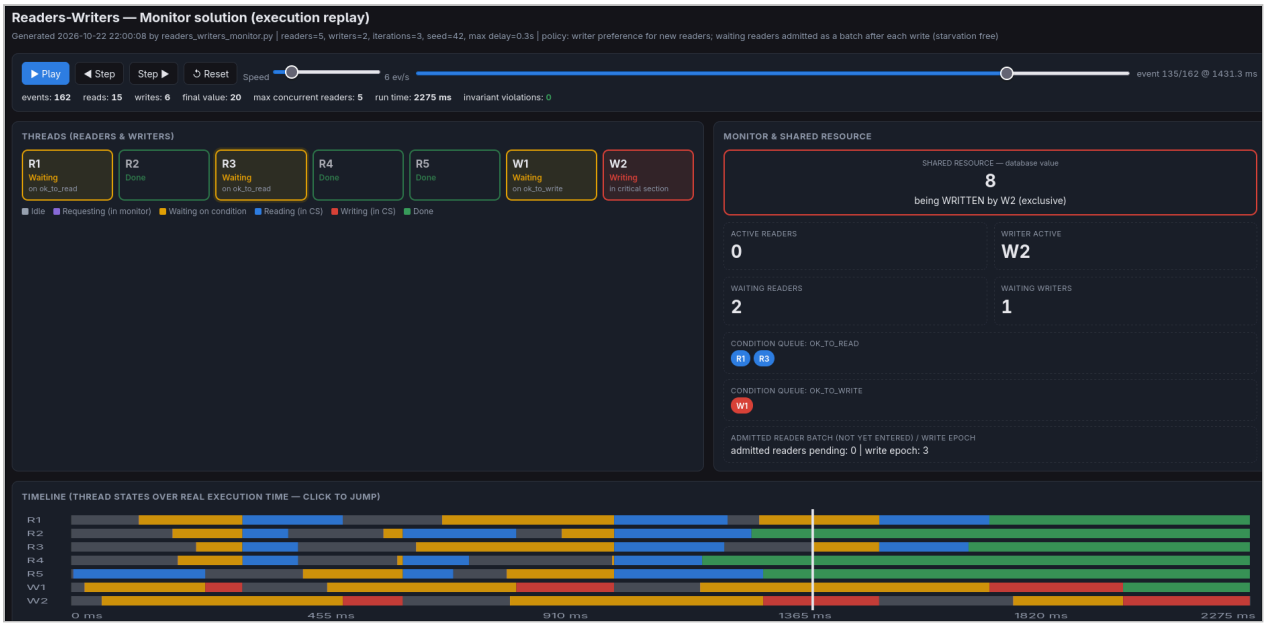


Figure 2. Readers–Writers (monitor) simulation

```
[1328.10 ms] R1 wait      wait(ok_to_read): writer(s) W2,W1 waiting
[1335.64 ms] R5 signal    signal(ok_to_write) -> wake one of W2,W1
[1335.79 ms] W2 enter_cs  ENTER critical section (writing, exclusive)
[1431.31 ms] R3 wait      wait(ok_to_read): writer W2 is writing
Summary: reads 15/15, writes 6/6, max concurrent readers 5, violations 0
```

7.3 Dining Philosophers Using Semaphore

The round table shows what state each philosopher is in, and each fork is drawn beside the philosopher who owns it. In this screenshot P3 is holding F0 and has just picked up F3, while P1 is hungry and holding F1. The trace shows P3 taking F0 before F3 because of the ordering rule, and then waiting for F3 while P2 is eating with it.

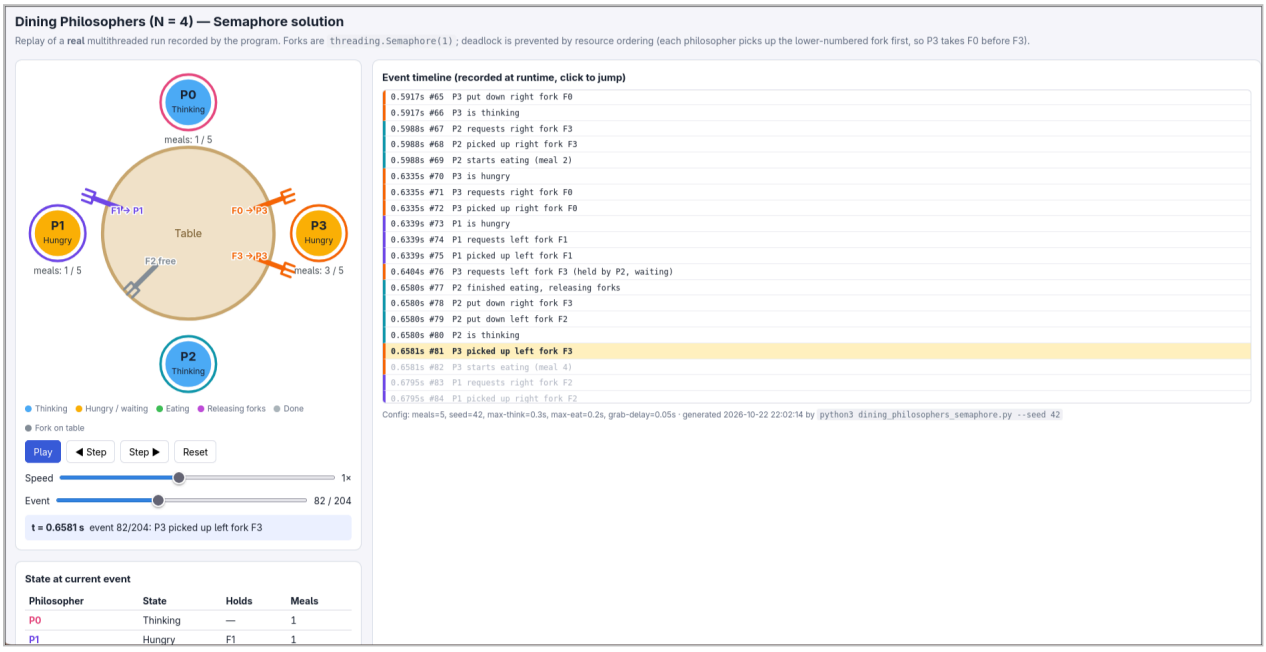


Figure 3. Dining Philosophers (semaphore) simulation

```
[0.6335s] P3 picked up right fork F0
[0.6339s] P1 picked up left fork F1
[0.6404s] P3 requests left fork F3 (held by P2, waiting)
[0.6580s] P2 put down right fork F3
[0.6581s] P3 picked up left fork F3
Result: all 4 threads terminated (no deadlock), meals 5/5/5/5
```

7.4 Dining Philosophers Using Monitor

P2 is eating with F2 and F3 while everyone else is thinking. A little earlier in the trace, P0 finished eating and signalled both of its waiting neighbours. P1 and P3 then ate at the same time, which is allowed because they don't sit next to each other. When P3 was done, it passed F2 and F3 on to P2.

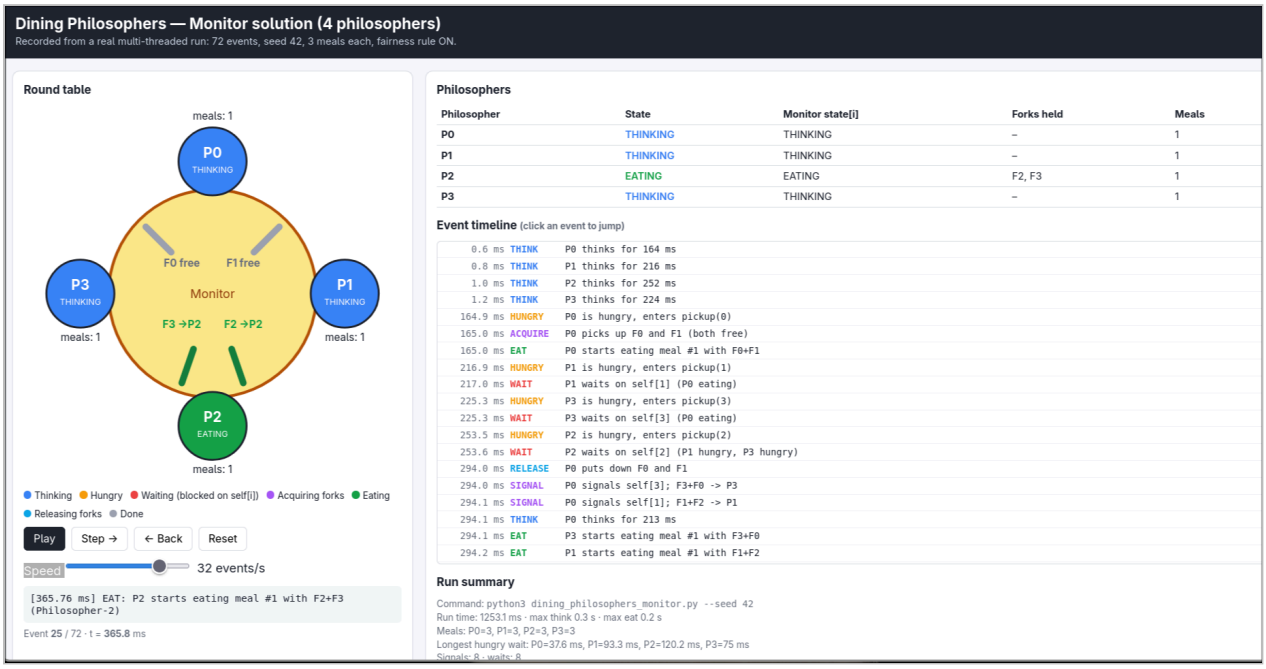


Figure 4. Dining Philosophers (monitor) simulation

```
[294.04 ms] SIGNAL  P0 signals self[3]; F3+F0 -> P3
[294.07 ms] SIGNAL  P0 signals self[1]; F1+F2 -> P1
[294.13 ms] EAT     P3 starts eating meal #1 with F3+F0
[294.19 ms] EAT     P1 starts eating meal #1 with F1+F2
[365.71 ms] SIGNAL  P3 signals self[2]; F2+F3 -> P2
[365.76 ms] EAT     P2 starts eating meal #1 with F2+F3
Result: meals 3/3/3/3, signals/waits 8/8, violations 0
```

Observation: using the same seed gives the same random delays, but the operating system decides the exact order the threads run in, so two runs can come out slightly different. Each HTML page always replays the run that produced it. Screenshots of the console output from all four runs are in `screenshots/`.

8. Synchronization Concepts Demonstrated

8.1 Process and Thread Synchronization

Synchronization controls the order in which threads use shared data, so the result doesn't depend on how lucky the scheduling was [1][2]. In all four of my programs, the only things that decide when a thread can continue are the semaphores or the monitor.

8.2 Race Condition

A race condition is when the result depends on how the steps of different threads interleave. In the writers' read-modify-write, an update would get lost if two writers overlapped. With the forks, two neighbours could grab the same fork. When the synchronization was taken out in test runs, the checkers reported these errors straight away.

8.3 Semaphore

A semaphore is an integer counter with two atomic operations. `acquire()` (wait, or P) blocks the thread while the counter is 0 and then lowers it by one. `release()` (signal, or V) raises it by one and may wake a waiting thread [1][8].

8.4 Monitor

A monitor is a module whose procedures always run under mutual exclusion and which uses condition variables for waiting. In Python I built one from a single `Lock` plus `Condition` objects that share that lock [5][7][8].

8.5 Mutual Exclusion

Mutual exclusion means no more than one thread is in a critical section at any moment. In my programs, a writer always writes alone, and a fork never has more than one owner.

8.6 Deadlock and Starvation

For deadlock to happen, four conditions must all hold at the same time: mutual exclusion, hold-and-wait, no preemption and circular wait. In the semaphore solution, resource ordering removes circular wait. In the monitor solution, picking up both forks atomically removes hold-and-wait [1][6]. Starvation is a different risk, where a thread keeps losing out to others. I reduced it with the service queue and batch admission in Readers–Writers, and with the ticket rule in the Dining Philosophers monitor.

9. Differences Between the Two Synchronization Methods

Aspect	**Semaphore**	**Monitor**
Basic idea	A counter, with `acquire()` and `release()` calls that the programmer places by hand	Shared data together with the only procedures allowed to use it, with mutual exclusion built in
Where the logic lives	Spread out over every reader, writer or philosopher	In one class (`ReadWriteMonitor`, `DiningMonitor`)
Waiting	Happens inside the semaphore, with no reason recorded	Happens on named condition variables, with the condition written out in a `while` loop
Typical mistakes	Calling `acquire()` in the wrong order (deadlock) or forgetting a `release()`	Writing `if` instead of `while` under Mesa semantics, or missing a signal
Fairness	Needs extra semaphores, like the service queue	Written as normal rules on the state (writer waiting, ticket order)
Dining Philosophers	One semaphore for each fork; ordering avoids deadlock	Both forks given out together; deadlock can't happen by design

Both approaches gave correct results with 0 violations. Semaphores fit naturally onto individual resources like forks. Monitors made the policy easier to read and change, and also easier to show visually, since each waiting thread records exactly what it is waiting for.

10. What Was Learned from the Assignment

The biggest thing I took away is that a concurrent program can look right and still break on some rare ordering of threads. So it isn't enough to argue from the design that it is correct. It also has to be checked while it runs, which is why every one of my programs counts invariant violations.

Solving the same problem with semaphores and then with a monitor made the difference between the two much clearer to me. With semaphores, the order of every `acquire()` and `release()` in every thread matters. With a monitor the rules sit in one class, but because of Mesa semantics each condition has to be checked again inside a `while` loop.

Dining Philosophers made the four deadlock conditions feel concrete. Breaking just one of them is enough to stop deadlock, either circular wait with fork ordering or hold-and-wait with atomic pickup. Starvation, on the other hand, is its own problem and needs its own rule. Replaying the recorded events in the browser let me actually watch readers sharing the resource, writers working alone and forks moving between neighbours, exactly as it happened in the run.

11. Disclosure of AI Assistance

As the course allows, I used Claude (Anthropic) as a programming assistant for this assignment. It generated the four Python implementations, the event recorder, the HTML replay pages and the runtime correctness checks, and it helped draft the README files and this report. The prompts I used are listed in `ai_prompts/prompts.md`.

My part was to give the assignment requirements and design constraints, pick the report format, review the generated code and output, and study the synchronization logic well enough to explain and modify it.

One problem turned up in the AI output. In the first version of the Dining Philosophers monitor page, the forks were drawn in the wrong places, even though the recorded data was correct. Comparing the page with the recorded snapshot showed the mismatch, and the drawing code was then fixed. Each correctness checker was also tested by switching off synchronization on purpose, to make sure a result of "0 violations" can be trusted.

12. References

1. Silberschatz, A., Galvin, P. B., and Gagne, G. *Operating System Concepts*, 10th Edition, Wiley, 2018. https://codex.cs.yale.edu/avi/os-book/OS10/
2. Tanenbaum, A. S., and Bos, H. *Modern Operating Systems*, 4th Edition, Pearson, 2014.
3. Downey, A. B. *The Little Book of Semaphores*, 2nd Edition, Green Tea Press, 2016. https://greenteapress.com/wp/semaphores/
4. Courtois, P. J., Heymans, F., and Parnas, D. L. "Concurrent Control with 'Readers' and 'Writers'", *Communications of the ACM*, 14(10), 1971.
5. Hoare, C. A. R. "Monitors: An Operating System Structuring Concept", *Communications of the ACM*, 17(10), 1974.
6. Dijkstra, E. W. "Hierarchical Ordering of Sequential Processes", *Acta Informatica*, 1(2), 1971.
7. Lampson, B. W., and Redell, D. D. "Experience with Processes and Monitors in Mesa", *Communications of the ACM*, 23(2), 1980.
8. Python Software Foundation. *threading — Thread-based parallelism*. https://docs.python.org/3/library/threading.html
9. Python Software Foundation. *threading — Semaphore Objects and Condition Objects*. https://docs.python.org/3/library/threading.html#semaphore-objects
10. Oracle. *The Java Tutorials: Concurrency — Synchronization*. https://docs.oracle.com/javase/tutorial/essential/concurrency/sync.html
11. Oracle. *The Java Tutorials: Intrinsic Locks and Synchronization*. https://docs.oracle.com/javase/tutorial/essential/concurrency/locksync.html
12. Oracle. *The Java Tutorials: Deadlock*. https://docs.oracle.com/javase/tutorial/essential/concurrency/deadlock.html
13. Oracle. *Java SE API Specification: java.util.concurrent.Semaphore*. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/Semaphore.html
14. MDN Web Docs. *SVG: Scalable Vector Graphics*. https://developer.mozilla.org/en-US/docs/Web/SVG

13. Academic Integrity Declaration

"I declare that I have personally tested and understood the submitted programs. I have disclosed the AI tools and external resources used in developing this assignment. I can explain the synchronization logic, generated HTML simulation, and modifications made to the code. I have not knowingly submitted another student's work or unacknowledged third-party material as my own."

Name: Felicia Fernandes

USN: NNM24IS083

Signature: _____

Date: _____